



Robotics & Computer Vision

Forward Kinematics: Manipulators & Mobile Robots

Week 7 — From joint/wheel inputs to end-effector and base pose — the foundational mapping every roboticist must master.

Week 7 Learning Objectives

1

Understand FK as a Function

Grasp the Forward Kinematics function conceptually: given joint angles or wheel velocities, compute the resulting pose of the TCP or mobile base.

2

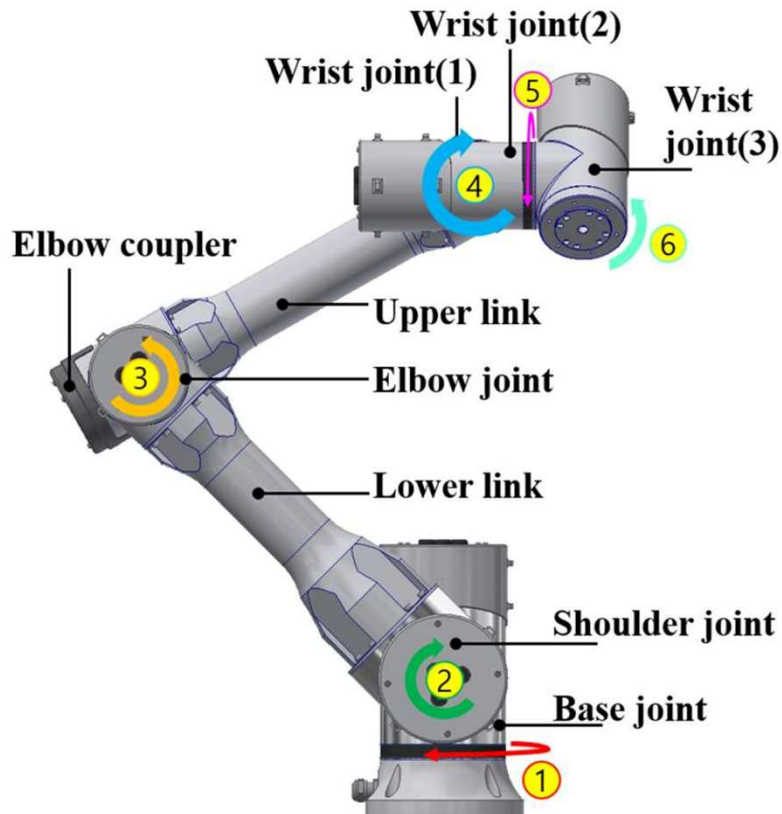
Apply FK in 2D and 3D

Work through planar 2R robots, full 6-DOF arms, and mobile robot models including unicycle and differential-drive configurations.

3

Use Robotics Toolbox & ETS

Leverage Python Robotics Toolbox and Elementary Transform Sequences to compute and visualize `fkine()` results with Swift.



How Week 7 Fits in the Course

- 1 — **Week 6**
SE(3), PoE, joint/task space, Jacobian — the kinematics framework
- 2 — **Week 7 ← You Are Here**
Deep dive: FK for manipulators and mobile base, extensive lab code
- 3 — **Week 8**
Inverse Kinematics — uses FK to verify IK solutions
- 4 — **Week 9**
Camera → base → IK → grasp: mapping AI/vision to robot action

Why Does Forward Kinematics Matter?

FK as a Measurement Function

FK reads encoder data and computes the current pose of the TCP or mobile base. Without it, the robot cannot know where it is in the world.

Foundation for Everything Else

- **Inverse Kinematics (Week 8):** FK is used to verify IK solutions.
- **Control loops:** FK converts encoder readings into pose feedback.
- **Collision checking:** requires knowing each link's pose.
- **Simulation & visualization:** simulators draw robots using FK.

⚠ Without a firm grasp of FK, Weeks 8–12 content will feel abstract and difficult to debug.

Review: SE(3) and Product of Exponentials

Week 6 introduced the mathematical backbone. A rigid-body pose in 3D is encoded as a homogeneous transformation matrix:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

Composing Transforms

For a serial chain, each link adds a transform. The total FK is their product:

$$T_0^n = T_0^1 T_1^2 \dots T_{n-1}^n$$

Product of Exponentials (PoE)

Choose a zero configuration M and screw axes $\{S_i\}$. The end-effector pose is:

$$T(\theta) = e^{[S_1]\theta_1} \dots e^{[S_n]\theta_n} M$$

Elegant and frame-placement-free — preferred in this course.

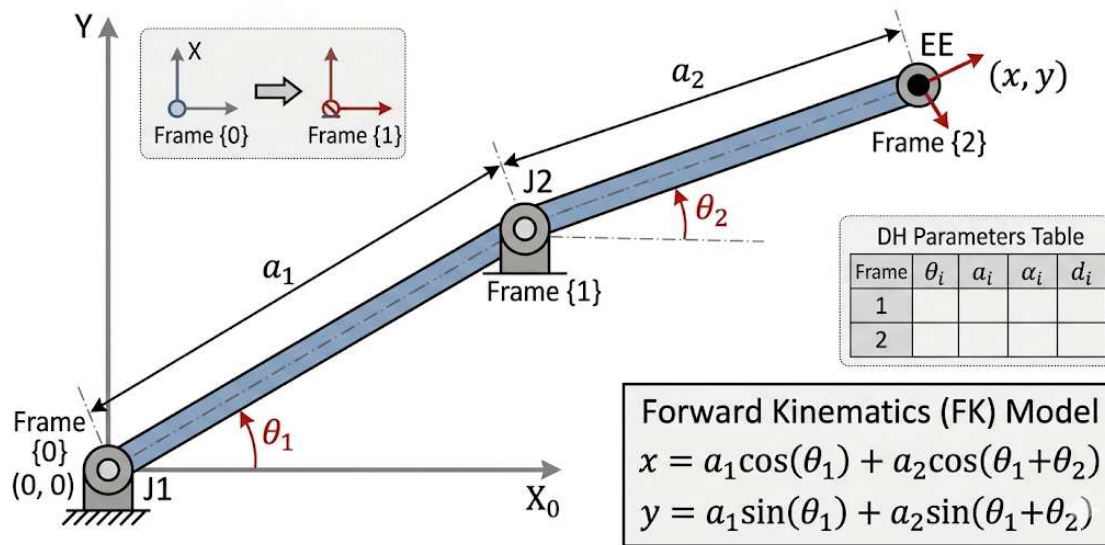
PART A

FK for Manipulators

Geometric intuition before the math — building from 2D planar robots upward.

Planar 2R Robot — The Model

Planar Two-Link Revolute (2R) Robot



Configuration

Two revolute joints operating in the XY plane — analogous to a human shoulder and elbow.

- Link lengths: L_1 (upper arm) and L_2 (forearm)
- Joint variables: θ_1 (shoulder angle), θ_2 (elbow angle relative to link 1)
- Output: TCP position (x, y) in the base frame

This is the simplest non-trivial manipulator — a perfect first FK exercise before moving to 3D.

FK Formula for the Planar 2R Robot

Applying trigonometry and frame composition, the TCP position is:

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$$

$$y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

Joint 1 Effect

θ_1 sets the global orientation of the entire arm. It rotates the whole kinematic chain around the base origin.

Joint 2 Effect

θ_2 bends the forearm relative to the upper arm, extending or retracting the TCP along a circular arc centered at the elbow.

The angle $(\theta_1 + \theta_2)$ is the absolute orientation of link 2 in the world frame — a key insight for intuition.

Visualizing TCP Behavior

Fix θ_1 , Sweep θ_2

TCP traces a **circular arc** centered at the elbow joint. Radius equals L_2 . This shows the isolated effect of the second joint.

Fix θ_2 , Sweep θ_1

TCP traces a **full circle** centered at the base origin. Radius equals $\sqrt{L_1^2 + L_2^2 + 2L_1L_2\cos\theta_2}$.

Sweep Both Jointly

TCP explores the full **annular workspace**: reachable from radius $|L_1 - L_2|$ to $L_1 + L_2$. Reveals singular boundaries.

These three thought experiments give students a visceral understanding of how each joint contributes independently and jointly to the final TCP position.

Numerical Example — 2R FK in Python

```
import numpy as np

def fk_2r(l1, l2, th1, th2):
    x = l1*np.cos(th1) + l2*np.cos(th1 + th2)
    y = l1*np.sin(th1) + l2*np.sin(th1 + th2)
    return x, y

l1 = l2 = 0.5 # link lengths in meters
th1 = np.deg2rad(30) # shoulder: 30°
th2 = np.deg2rad(45) # elbow: 45°

x, y = fk_2r(l1, l2, th1, th2)
print(f"TCP position: x={x:.4f} m, y={y:.4f} m")
```

Expected Output

With $l_1 = l_2 = 0.5$, $\theta_1 = 30^\circ$, $\theta_2 = 45^\circ$.

$$x \approx 0.683 \text{ m}, \quad y \approx 0.604 \text{ m}$$

Lab 7.1 Task

Use this function to plot the full TCP workspace by sweeping θ_1 , θ_2 in $[-\pi, \pi]$ and collecting (x, y) scatter points.

Limitations of the 2R Example

2D Only — No Orientation

The formula outputs only (x, y) . Real tasks require a full 6D pose: position (x, y, z) plus orientation $(R \text{ in } SO(3))$.

No Out-of-Plane Motion

Gravity, joint offsets, and 3D link geometries are entirely absent. Extending to 3D requires homogeneous matrices, not just trigonometry.

Generalization via SE(3)

The correct generalization is a chain of *4 times 4* SE(3) transforms, one per joint. This is what DH parameters and ETS formalize.

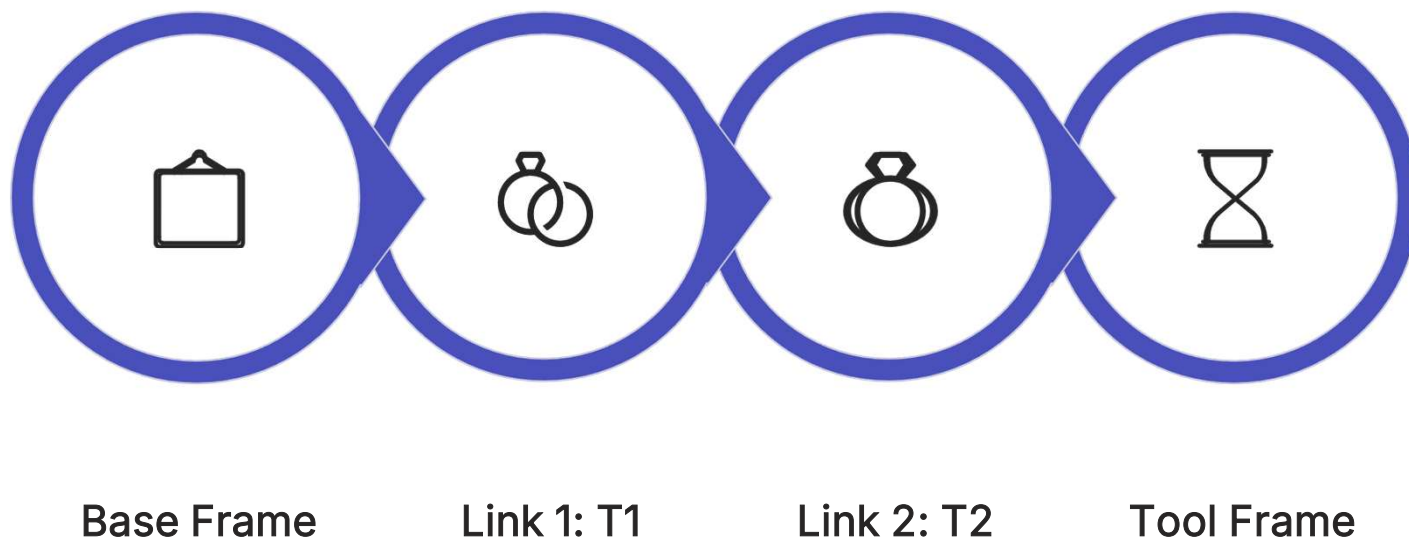
PART B

3D FK: Transform Chains & ETS

Extending planar FK to full 3D using systematic transform composition and the Elementary Transform Sequence framework.

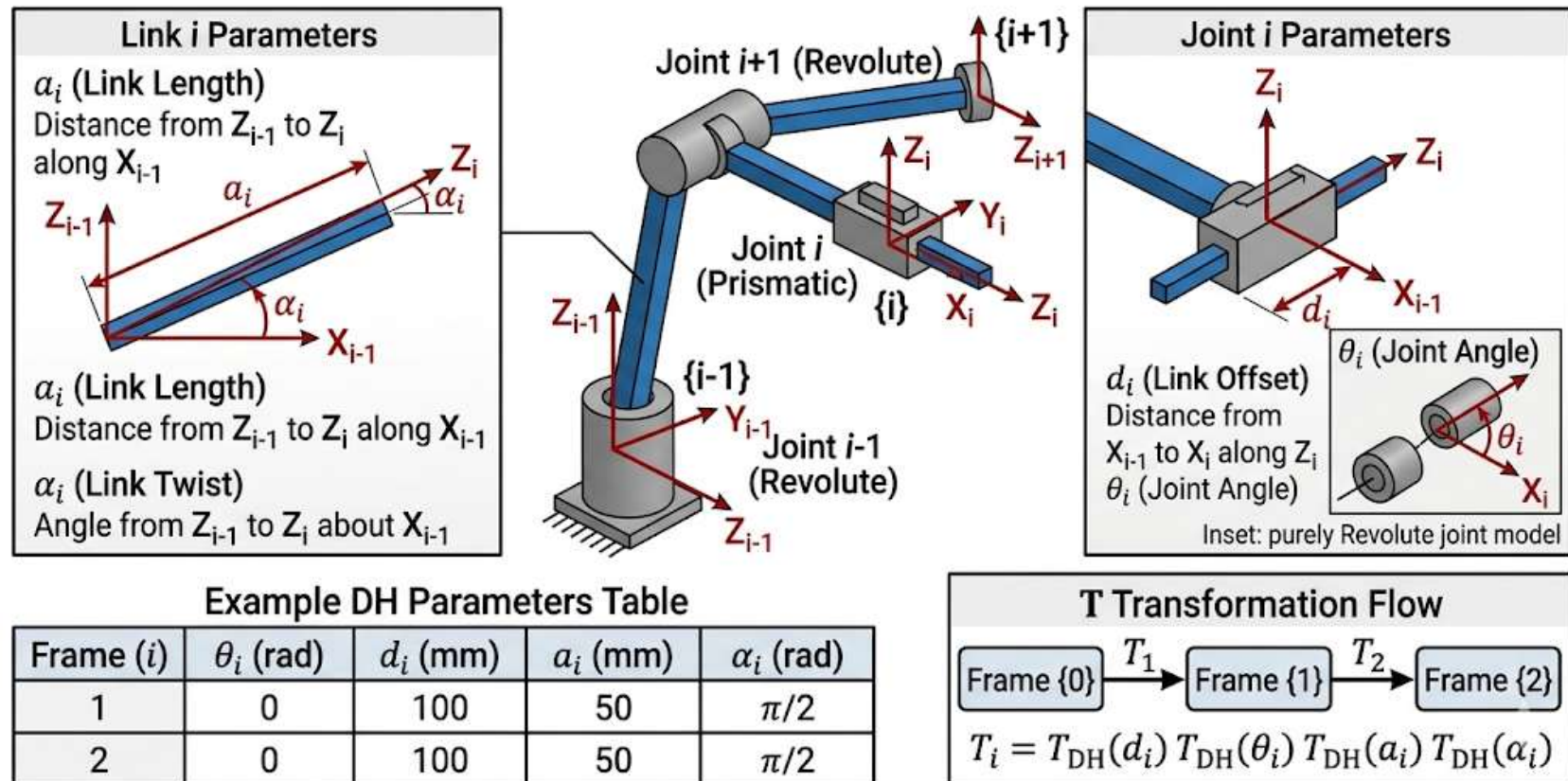


3D Kinematic Chain Structure



Each joint contributes one transform $T_{i-1}^i(\theta_i)$ that depends on the joint variable. The complete FK is the ordered product of all these transforms, mapping the base frame to the tool frame.

Denavit–Hartenberg Parameters — Quick Recap



Why Use ETS / PoE Instead of DH?

DH: Powerful but Fragile

- Requires careful frame placement at every joint
- Small frame errors propagate into large FK errors
- Parallel joints and offset axes are tricky to handle

ETS / PoE: Direct and Robust

- Describe each joint directly as a rotation or translation primitive
- No auxiliary frame placement — use the zero-pose geometry
- Natively supported in Python Robotics Toolbox
- Easier for AI/CS students to reason about programmatically

✓ This course (RCV) actively uses ETS + PoE to reduce formula overhead and focus on concepts.

Elementary Transform Sequence (ETS)

An ETS is an ordered list of primitive transforms. Each primitive is one of the six elementary types:

$R_x(|theta)$

Rotation about X-axis

$R_y(|theta)$

Rotation about Y-axis

$R_z(|theta)$

Rotation about Z-axis — most common for revolute

$T_x(d)$

Translation along X

$T_y(d)$

Translation along Y

$T_z(d)$

Translation along Z — most common for Prismatic joints

The joint variable q_i is inserted into exactly one primitive in the ETS. All other primitives have fixed values determined by the link geometry.

ETS Example — Planar 2R via Robotics Toolbox

```
import roboticstoolbox as rtb
import numpy as np

# Load the built-in planar 2-link ETS model
robot = rtb.models.ETS2.planar2()

# Define joint configuration
q = np.array([0.2, 0.5]) # theta1=0.2 rad, theta2=0.5 rad

# Compute FK
T = robot.fkine(q)

print(robot)
print("FK result T =\n", T.A)
print("TCP position:", T.t)
print("TCP rotation:\n", T.R)
```

 Lab 7.2 compares this output with the manually coded `fk_2r()` function. Perfect match validates both implementations.

Introduction to Manipulator Kinematics

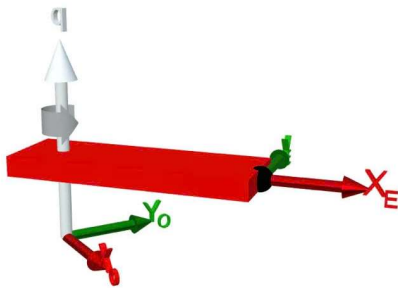
- Contemporary manipulators consist of rigid links connected by joints, typically revolute or prismatic. Motors control joint motion to achieve desired end-effector tasks. The manipulator's geometry determines its kinematic behavior and workspace capabilities.
- Traditional approaches use **Denavit-Hartenberg parameters**, but we'll focus on the more intuitive **twist coordinate** representation that provides superior geometric insight and computational advantages.



Joint Types and Mechanisms

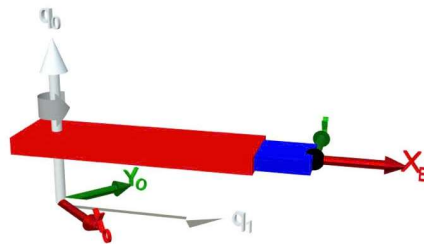
Revolute Joints

Revolute joints provide rotational motion about a fixed axis. They are the most common joint type in robotic manipulators, offering precise angular control and reliable operation. Joint angles typically range from $-\pi$ to π radians.



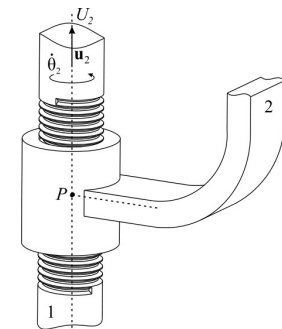
Prismatic Joints

Prismatic joints enable linear translation along a directed axis. They provide straight-line motion and are essential for applications requiring linear positioning or extending reach capabilities.

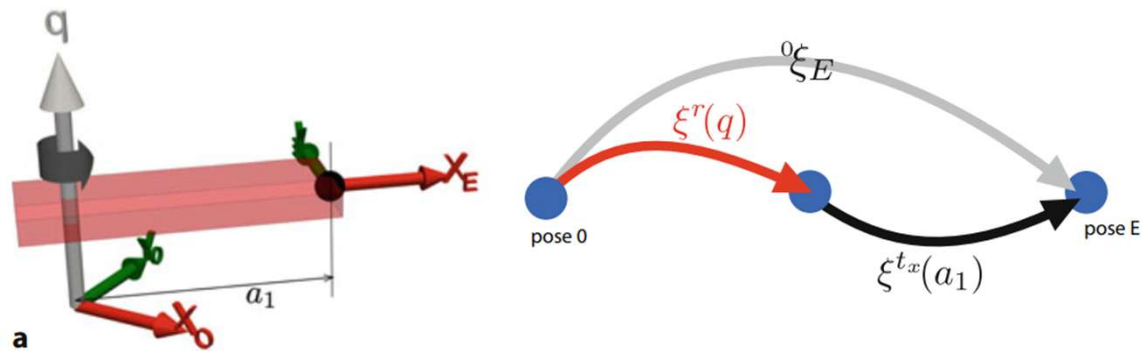


Helical Joints

Helical joints combine rotation and translation in a screw-like motion. Though less common, they appear in specialized applications requiring coupled rotational and linear motion.



2-Dimensional (Planar) Robotics Arms



We begin with simple planar robots operating in 2D space. Consider a basic single-joint robot like a clock hand - it comprises one revolute joint and one link. The pose graph shows composition of two relative poses: rotation by joint angle q , then fixed translation in x-direction by distance a_1 .

01

Single Joint Robot

Rotation by joint angle q followed by translation distance a_1 in x-direction

02

Pose Composition

End effector pose: ${}^0\xi_e = \xi^r(q) \oplus \xi^{tx}(a_1)$

Elementary Transformation Sequences

Using the Robotics Toolbox, we represent kinematics through Elementary Transformation Sequences (ETS). For 2D cases, we use ET2 class methods to create transformations and compose them with the `*` operator.



ET2.R()

Creates variable rotation transformation

ET2.tx(a1)

Creates constant translation in x-direction

Composition

`e = ET2.R() * ET2.tx(1)` creates ETS2 instance

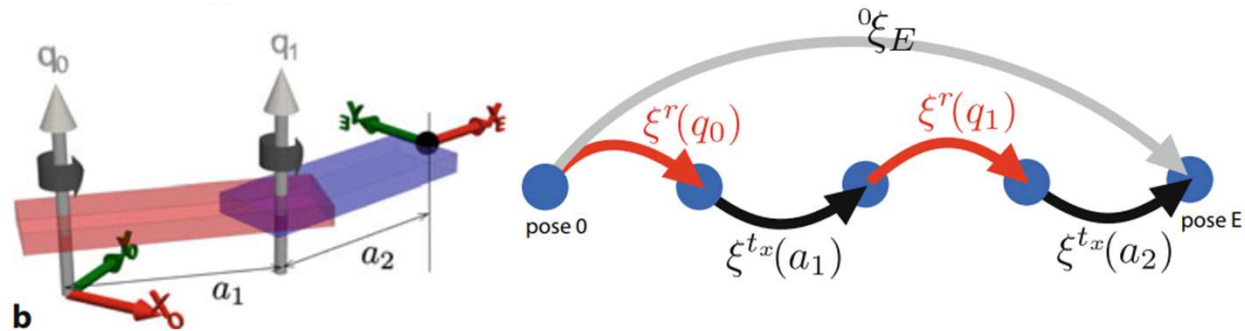
The result is a list-like object containing two rigid-body transforms: one variable and one constant. For joint angle $q = \pi/6$ radians, we compute the end-effector pose using **`e.fkine(pi/6)`**.

Two-Joint Planar Robot

A more capable robot has two revolute joints q_0 and q_1 . From the pose graph, the end effector pose becomes: ${}^0\xi_e = \xi^r(q_0) \oplus \xi^{tx}(a_1) \oplus \xi^r(q_1) \oplus \xi^{tx}(a_2)$

Robot Characteristics

- Most positions reachable with two different joint angle sets
- Can position end effector anywhere within reach
- Cannot achieve arbitrary orientation



Toolbox Implementation

`a1 = 1; a2 = 1;`

`e = ET2.R() * ET2.tx(a1) * ET2.R() * ET2.tx(a2)`

This creates sequential joint variables q_0 and q_1 . We evaluate for specific angles: `e.fkine(np.deg2rad([30, 40]))`

3-Dimensional Robotic Arms

Real-world manipulators operate in 3D with task space $T \subset \mathbb{R}^3 \times (S^1)^3$, allowing arbitrary position and orientation within their workspace.



6-Axis Industrial Robot

Standard configuration providing full 6 DoF positioning and orientation capability



Human-like arm

2 DoF spherical shoulder, upper arm, elbow, lower arm, 3 DoF spherical wrist



PUMA 560: The Research Standard

The Programmable Universal Manipulator for Assembly (PUMA) 560, released in 1978, became the "white rat" of robotics research. It featured anthropomorphic design, electric motors, and spherical wrist - the archetype for modern industrial robots.

1

1978 Release

First modern industrial robot with anthropomorphic design

2

Research Impact

Catalyzed 1980s robotics research, became common lab robot

3

Legacy

Well-studied parameters make it ideal for examples and benchmarks

The PUMA's anthropomorphic design approximated human worker dimensions and reach. Roboticists adopted human terminology: waist, shoulder, elbow, wrist for joints 0, 1, 2, and 3-5 respectively.

3-Dimensional Robotic Arms

Real-world manipulators operate in 3D with task space $T \subset \mathbb{R}^3 \times (S^1)^3$, allowing arbitrary position and orientation within their workspace.



6-Axis Industrial Robot

Standard configuration providing full 6 DoF positioning and orientation capability



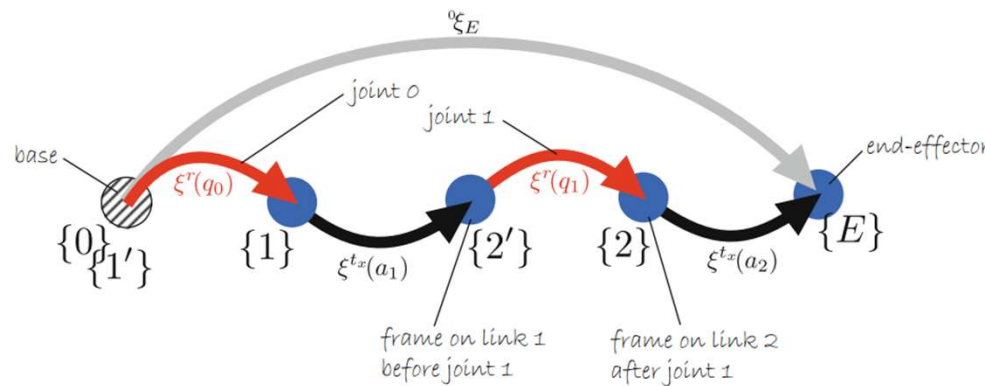
Human-like arm

2 DoF spherical shoulder, upper arm, elbow, lower arm, 3 DoF spherical wrist

Forward Kinematics as Link Chains

Robot structure consists of links connected by joints. Link ℓ frame pose depends on joint coordinates $q_j, j \in [0, \ell)$. Each link is parent to the next and child to the previous. Forward kinematics becomes:

$${}^0\xi_N = {}^0\xi_1(q_0) \oplus {}^1\xi_2(q_1) \oplus \dots \oplus {}^{N-1}\xi_N(q_{N-1})$$



01

Link Frames

Each link has coordinate frame, pose depends on preceding joint variables

02

Parent-Child Relations

Link ℓ is parent of link $\ell+1$, child of link $\ell-1$

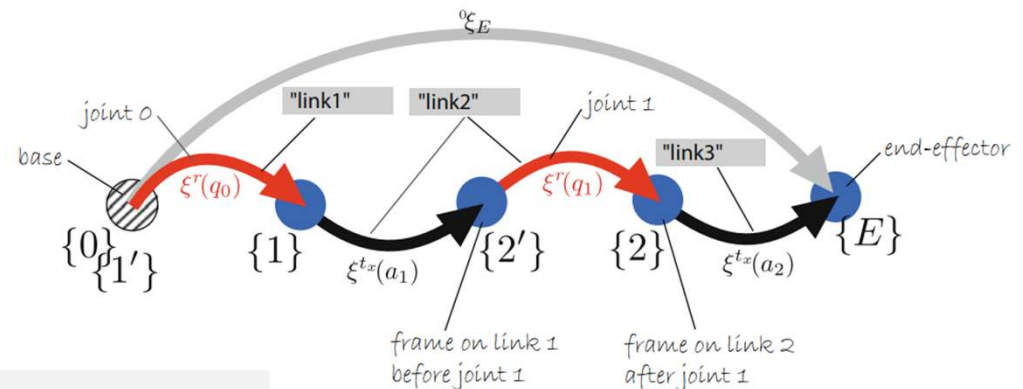
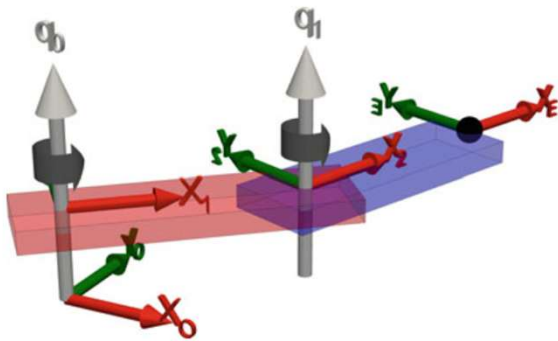
03

Composition

Forward kinematics: $K(q)$ composes all relative transformations

2-Dimensional (Planar) Case

Frame $\{l'\}$ is the parent side of the joint, that is the joint is a transformation from $\{l'\}$ on the parent link to frame $\{l\}$ on the child. When $q_{l-1} = 0$ these two frames are identical



```
a1 = 1; a2 = 1
link1 = Link2(ET2.R(), name="link1")
link2 = Link2(ET2.tx(a1) * ET2.R(), name="link2", parent=link1)
link3 = Link2(ET2.tx(a2), name="link3", parent=link2)
robot = ERobot2([link1, link2, link3], name="my robot")
```

2-Dimensional (Planar) Case

We pass a list of these links objects to the robot object constructor and its display shows the rich structure we have created.

```
>>> robot = ERobot2([link1, link2, link3], name="my robot")
```

```
ERobot2: my robot, 2 joints (RR)
```

link	link	joint	parent	ETS: parent to link
0	link1	0	BASE	$R(q_0)$
1	link2	1	link1	$tx(1) \oplus R(q_1)$
2	@link3		link2	$tx(1)$

At the top of the table we see the class type, the robot's name, the number of joints, and the joint structure. The @ symbol indicates a link frame that is also an end effector – it has no child links.

2-Dimensional (Planar) Case

We pass a list of these links objects to the robot object constructor and its display shows the rich structure we have created.

```
>>> robot = ERobot2([link1, link2, link3], name="my robot")
```

```
ERobot2: my robot, 2 joints (RR)
```

link	link	joint	parent	ETS: parent to link
0	link1	0	BASE	$R(q_0)$
1	link2	1	link1	$tx(1) \oplus R(q_1)$
2	@link3		link2	$tx(1)$

At the top of the table we see the class type, the robot's name, the number of joints, and the joint structure. The @ symbol indicates a link frame that is also an end effector – it has no child links.



3-Dimensional Robotic Arms

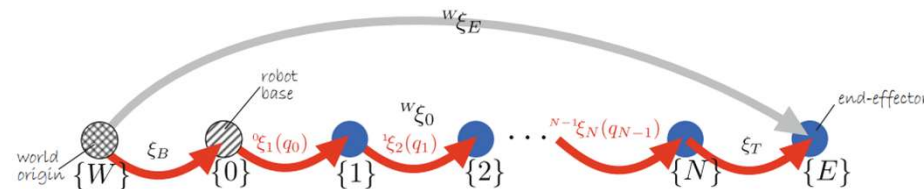
Most real-world robot manipulators have a task space $T \subset \mathbb{R}^3 \times (S^1)^3$ allowing arbitrary position and orientation of the end effector within its 3D working envelope. This requires a robot with configuration space $\dim C \geq \dim T$, achieved by six or more joints.

```
a1 = 1; a2 = 1e = ET.Rz() * ET.Ry() * ET.tz(a1) * ET.Ry()  
* ET.tz(a2) * ET.Rz() * ET.Ry() * ET.Rz()
```

ERobot: noname, 6 joints (RRRRRR)

link	link	joint	parent	ETS: parent to link
0	link0	0	BASE	Rz (q0)
1	link1	1	link0	Ry (q1)
2	link2	2	link1	tz (1) $\oplus$ Ry (q2)
3	link3	3	link2	tz (1) $\oplus$ Rz (q3)
4	link4	4	link3	Ry (q4)
5	@link5	5	link4	Rz (q5)

Tool and Base Transformation



Practical robots require base and tool transformations extending the kinematic chain:

$${}^W\xi_E = \underbrace{{}^W\xi_0}_{\xi_B} \oplus {}^0\xi_1 \oplus {}^1\xi_2 \oplus \dots \oplus {}^{N-1}\xi_N \oplus \underbrace{{}^N\xi_E}_{\xi_T}$$

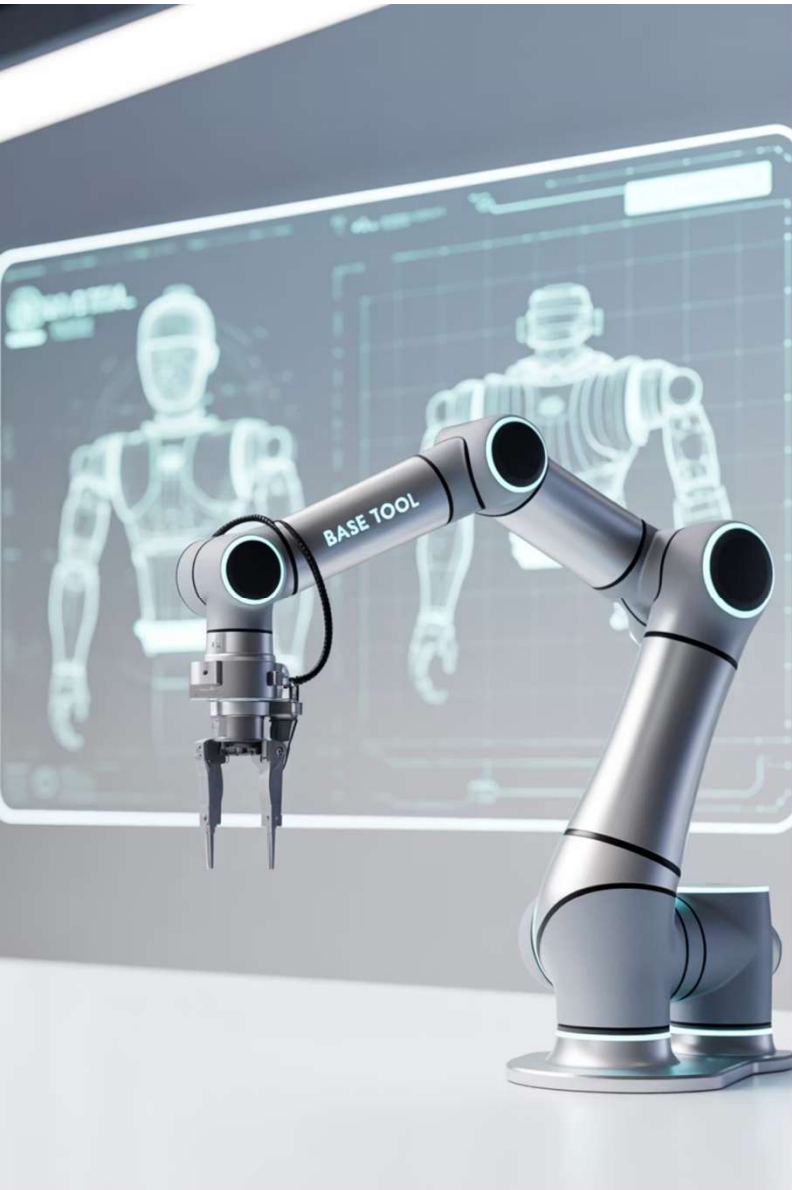
Base Transformation ξ_B

Places robot base at arbitrary pose in world frame. Fixed for manufacturing cells, time-varying for mobile robots.

Tool Transformation ξ_T

Describes tool tip pose relative to final link frame. Varies by application: gripper, screwdriver, welding torch.

Convention: robot tool points in z-direction. Tool transformation might include multiple components - tool holder plus specific tool.



Coordinate Frames and Configuration

Robot kinematics requires establishing coordinate frames to describe spatial relationships. The **base frame** S attaches to the manipulator's stationary base, while the **tool frame** T moves with the end-effector.

The forward kinematics problem seeks the transformation $g_{st}(\theta)$ that maps joint angles θ to the tool frame configuration relative to the base frame. This mapping belongs to the special Euclidean group $SE(3)$, representing both position and orientation in 3D space.

Twist Coordinates Foundation

Geometric Representation

- Twist coordinates provide an elegant geometric framework for describing joint motions.
- Each joint motion generates a twist ξ that characterizes the instantaneous screw motion about the joint axis.

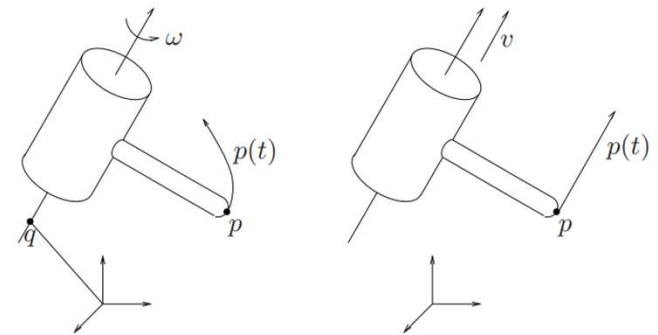
For revolute joints, the twist has the form:

$$\xi = \begin{bmatrix} -\omega \times \mathbf{q} \\ \omega \end{bmatrix} \quad \text{where } \omega \text{ is the unit axis direction and } \mathbf{q} \text{ is any point on the axis.}$$

Prismatic joints generate pure translation, represented by:

$$\xi = \begin{bmatrix} \mathbf{v} \\ 0 \end{bmatrix} \quad \text{where } \mathbf{v} \text{ is the unit direction vector of translation.}$$

This unified representation allows consistent treatment of all joint types within the same mathematical framework.



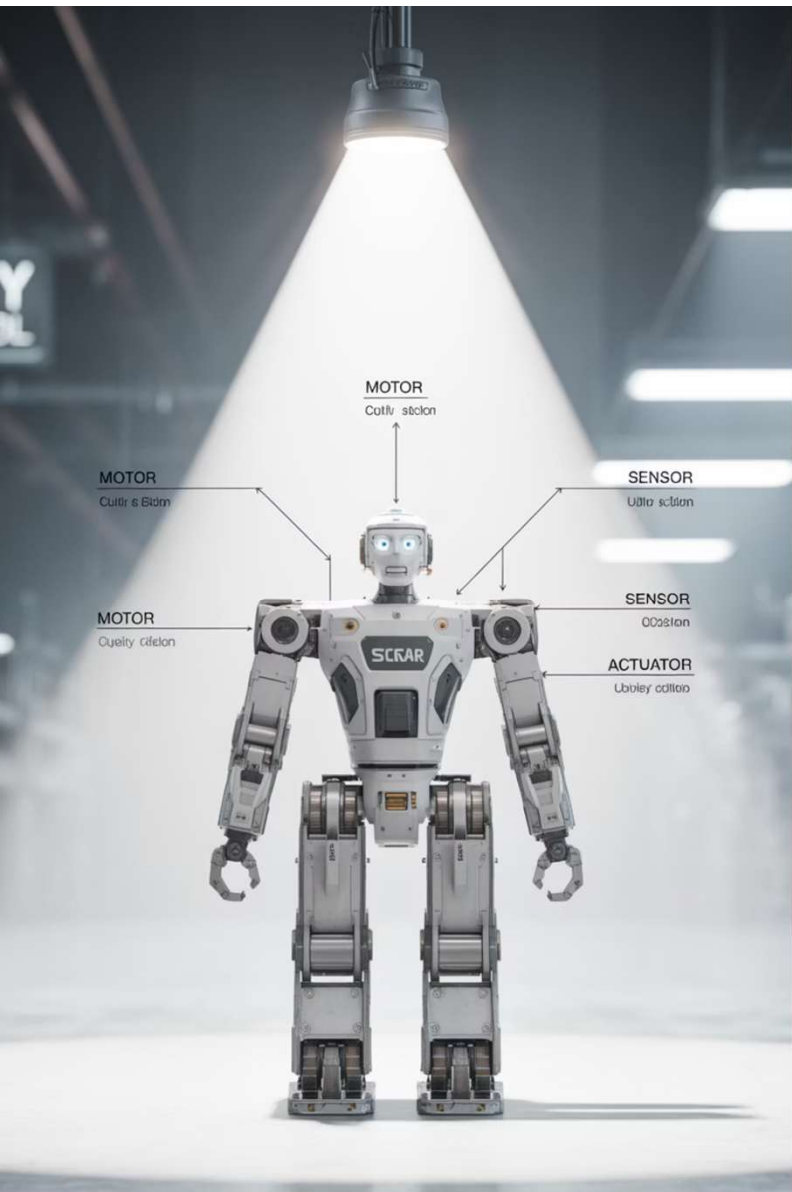


Product of Exponentials Formula

For a general robot we can write the forward kinematics in product of exponential (PoE) form as

$$\mathbf{T}_E = (e^{q_1[\hat{\mathbf{s}}_1]} \dots e^{q_N[\hat{\mathbf{s}}_N]}) \mathbf{T}_E(0)$$

where $\mathbf{T}_E(0)$ is the end-effector pose when the joint coordinates are all zero and $\hat{\mathbf{s}}_i$ is the unit twist for joint j expressed in the world frame at that configuration. This can also be written as



SCARA Robot Example - Setup

Consider a SCARA (Selective Compliance Assembly Robot Arm) manipulator with four joints: three revolute and one prismatic. This configuration provides excellent performance for assembly tasks requiring precise horizontal positioning.

Joint Configuration

- Joint 1: Base rotation (revolute)
- Joint 2: Arm rotation (revolute)
- Joint 3: Forearm rotation (revolute)
- Joint 4: Vertical translation (prismatic)

Reference Configuration

We establish $\theta = 0$ as the fully extended configuration with the tool frame coincident with the base frame. This choice simplifies the initial transformation $g_{st}(0) = I$.

SCARA Robot – Forward Kinematics

Computing the SCARA robot's forward kinematics begins with determining joint twists in the reference configuration:

To construct the twists for the revolute joints, note that

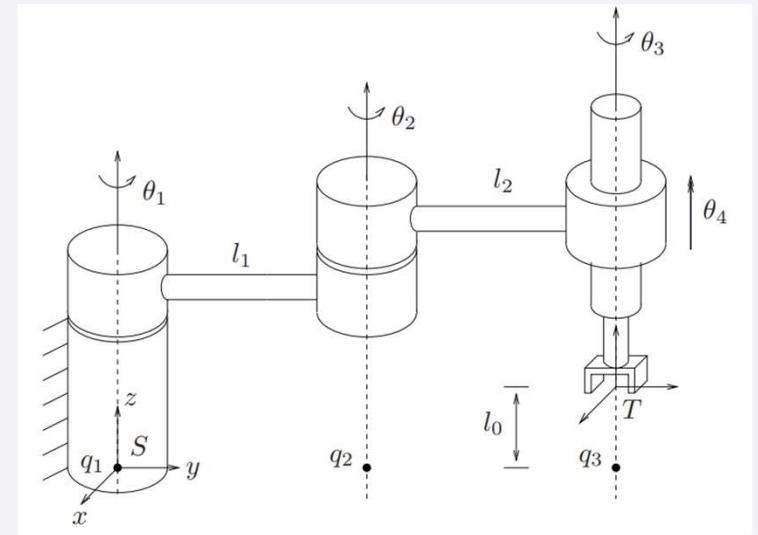
$$\omega_1 = \omega_2 = \omega_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

and we can choose axis points

$$q_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad q_2 = \begin{bmatrix} 0 \\ l_1 \\ 0 \end{bmatrix} \quad q_3 = \begin{bmatrix} 0 \\ l_1 + l_2 \\ 0 \end{bmatrix}$$

This yields twists

$$\xi_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \xi_2 = \begin{bmatrix} l_1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \xi_3 = \begin{bmatrix} l_1 + l_2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \xi_4 = \begin{bmatrix} v_4 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$



The forward kinematics map of the manipulator has the form

$$T_E = \left(e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} e^{\hat{\xi}_3 \theta_3} e^{\hat{\xi}_4 \theta_4} \right) T_E(0)$$

Python Implementation

```
import numpy as np
from spatialmath import SE3, Twist3
from roboticstoolbox import PoERevolute, PoEPrismatic, robot

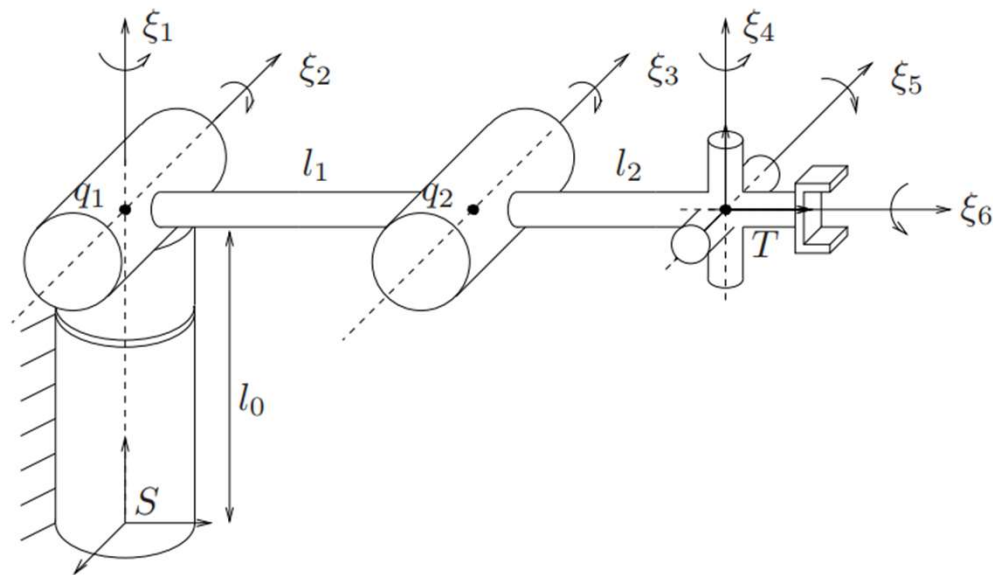
# Thông số robot SCARA
L1 = 0.4 # Độ dài link 1 (m)
L2 = 0.3 # Độ dài link 2 (m)
L3 = 0.2 # Khoảng cách từ khớp 3 đến end-effector (m)

# Bước 1: Xác định cấu hình tham chiếu T0 (khi tất cả góc khớp = 0)
# Khi tất cả khớp = 0, end-effector ở vị trí (0, L1+L2, L3)
T0 = SE3.Trans(0, L1 + L2, L3)

# Bước 2: Xác định các twists cho từng khớp tại cấu hình tham chiếu
omega1 = [0, 0, 1] # Trục quay z
q1 = [0, 0, 0] # Điểm trên trục (gốc tọa độ)
xi1_coords = PoERevolute(omega1, q1)
omega2 = [0, 0, 1] # Trục quay z
q2 = [0, L1, 0] # Điểm trên trục
xi2_coords = PoERevolute(omega2, q2)
omega3 = [0, 0, 1] # Trục quay z
q3 = [0, L1 + L2, 0] # Điểm trên trục
xi3_coords = PoERevolute(omega3, q3)
v4 = [0, 0, 1] # Hướng tịnh tiến z
xi4_coords = PoEPrismatic(v4)

# Bước 3: Kiểm tra với các góc khớp cụ thể
q = [np.pi/4, np.pi/6, np.pi/3, 0.1] # [45°, 30°, 60°, 0.1m]
# Phương pháp tính bằng Robot Class
SCARA = robot.PoERobot([xi1_coords, xi2_coords, xi3_coords, xi4_coords], T0)
TE = SCARA.fkine(q)
SCARA.fkine(q).printline()
```

Homework



Compute forward kinematics of this robot?

Visualization with Swift



What Swift Provides

- Interactive 3D rendering of the robot model in a browser
- Joint sliders: move each θ_i and watch TCP follow FK in real time
- Trajectory playback for animated motion sequences
- Visual confirmation that your ETS/URDF model is geometrically correct

✓ Swift builds geometric intuition before tackling IK — students "see" the FK function before inverting it.

PART C

6-DOF Arm FK

Yahboom & Panda — full spatial kinematics with position and orientation output.



Yahboom 6-DOF in the RCV Course

Hardware Context

The Yahboom 6-DOF arm is the physical platform for manipulation labs. Joint angles are read from servo encoders or set via software commands. The FK model then computes the real-time TCP pose.

Software Pipeline

1

Read Encoders

Servo feedback: $q \in \mathbb{R}^6$

2

FK Model

ETS / URDF chain

3

TCP Pose

$T \in SE(3)$

FK Example — Panda 7-DOF Arm

```
import roboticstoolbox as rtb
import numpy as np

# Load Panda from URDF (7-DOF, similar workflow to Yahboom 6-DOF)
robot = rtb.models.URDF.Panda()

configs = {
    "zero pose (qz)": robot.qz,
    "ready pose (qr)": robot.qr,
}

for name, q in configs.items():
    T = robot.fkine(q)
    print(f"\n--- {name} ---")
    print(f"  Joint angles (rad): {np.round(q, 3)}")
    print(f"  TCP position (m):    {np.round(T.t, 4)}")
    print(f"  TCP rotation:\n{np.round(T.R, 4)}")
```

 Yahboom uses an identical call pattern once its URDF or ETS model is loaded — same API, different model file.

Interpreting a 6D TCP Pose



Position

The Cartesian coordinates (x, y, z) of the tool center point in the robot's base frame.
Accessed via `T.t`.



Orientation

The 3×3 rotation matrix describing how the tool frame is oriented relative to the base.
Accessed via `T.R`.



Equivalent Representations

The same orientation can be expressed as Euler angles or unit quaternion $q = (q_w, q_x, q_y, q_z)$ for compact storage.

□ This 6D pose is exactly what IK (Week 8) will receive as a target and attempt to invert back to joint angles.

FK for Calibration & Verification

Why Calibration Matters

If the FK model has incorrect link lengths or joint offsets, the computed TCP pose will differ from the physical TCP position. This error propagates into every downstream task: grasping, path planning, and camera-to-robot mapping.

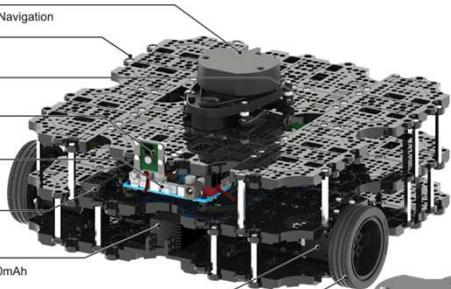
Calibration Workflow

1. Command the arm to a known pose (e.g., touching a calibration target)
2. Run FK to compute the predicted TCP position
3. Measure the actual TCP position (ruler or camera)
4. Adjust link parameters until prediction matches measurement

⚠ Always verify FK before camera-to-robot mapping (Week 9). A bad FK model makes vision-guided grasping impossible to debug.

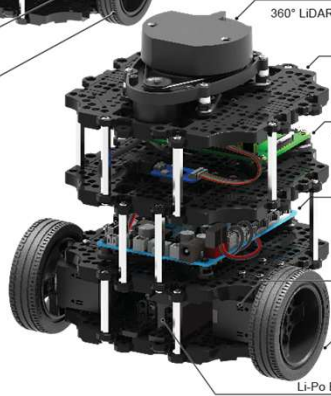
Waffle Pi

360° LiDAR for SLAM & Navigation
 Scalable Structure
 Single Board Computer (Raspberry Pi)
 Raspberry Pi Camera for Perception
 OpenCR (32-bit ARM® Cortex®-M7)
 Bluetooth Module for Remote Controller
 Li-Po Battery 11.1V 1,800mAh
 DYNAMIXEL x 2 for Wheels
 Sprocket Wheels for Tire and Caterpillar



Burger

360° LiDAR for SLAM & Navigation
 Scalable Structure
 Single Board Computer (Raspberry Pi 3)
 OpenCR (32-bit ARM® Cortex®-M7)
 DYNAMIXEL x 2 for Wheels
 Sprocket Wheels for Tire and Caterpillar
 Li-Po Battery 11.1V 1,800mAh

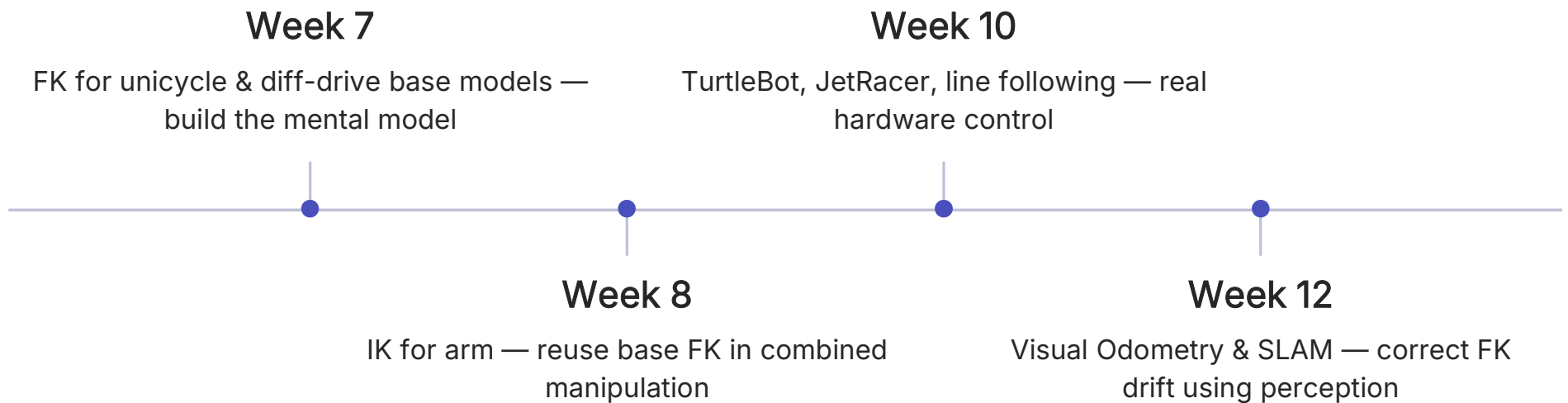


PART D

Mobile Robot FK

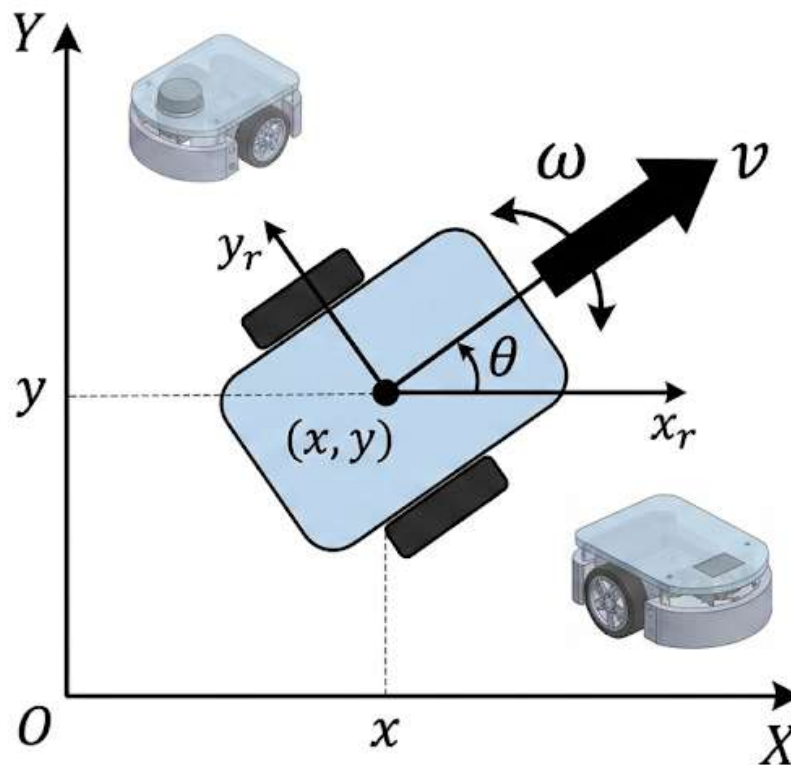
Unicycle, differential drive, and Ackermann — computing base pose from wheel commands.

Mobile Robots in the RCV Curriculum



The FK philosophy is identical across all platforms: **input is a control command, output is a base pose**. Mastering it in Week 7 makes the hardware labs in Weeks 10–12 much more tractable.

The Unicycle Model



Unicycle Model Kinematics

- **Robot State:**

(x, y, θ) - Position (x, y) and Orientation θ

- **Model Inputs & Parameters:**

v : Forward linear velocity

ω : Angular velocity (yaw rate)

r : Wheel radius

- **Kinematic Relations:**

$$\dot{x} = v \cdot \cos(\theta)$$

$$\dot{y} = v \cdot \sin(\theta)$$

$$\dot{\theta} = \omega$$

Discrete-Time Unicycle FK Step

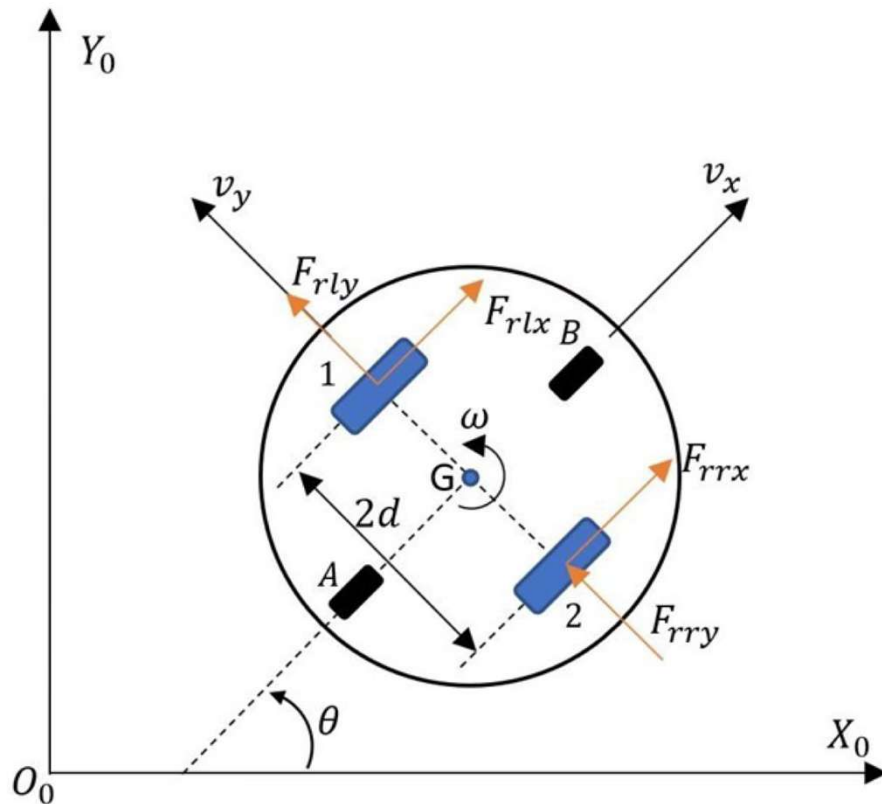
```
import numpy as np

def unicycle_step(x, y, th, v, w, dt):
    """
    Euler integration of unicycle kinematics.
    Inputs:  current pose (x,y,th), velocities (v,w), timestep dt
    Outputs: next pose (x_new, y_new, th_new)
    """
    x_new = x + v * np.cos(th) * dt
    y_new = y + v * np.sin(th) * dt
    th_new = th + w * dt
    return x_new, y_new, th_new
```

This Euler integration approximates the continuous ODE with step size Δt . Accuracy improves with smaller Δt . For Lab 7.6, use $\Delta t = 0.1s$ and run for 100 steps.

 Euler integration accumulates error over time — this is exactly the "drift" that VO/SLAM (Week 12) corrects.

Differential Drive Model



From Wheel Speeds to Unicycle Inputs

Given left/right wheel speeds v_L , v_R and wheel separation W :

$$V = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{W}$$

Feed (v, ω) into `unicycle_step()` — the same function handles both models. The only difference is how the inputs are derived from hardware commands.

📖 TurtleBot and most wheeled platforms at this level use differential drive. JetRacer uses Ackermann (Week 10).

Ackermann Steering (JetRacer)

Model Inputs

Unlike differential drive, Ackermann steering takes:

- Forward speed v
- Steering angle δ
- Wheelbase L

$$\omega \approx \frac{v \tan \delta}{L}$$

Approximation Strategy

For small steering angles, Ackermann can be approximated by the unicycle model with the computed ω . This is sufficient for Week 7 purposes.

Full Ackermann kinematics, including inner/outer wheel constraints, will be treated in **Week 10** alongside JetRacer lab sessions.

Summary: FK for Mobile Robots

Pose is Cumulative

Base pose (x, y, θ) is the time-integral of velocity commands. There is no absolute position sensor — only dead reckoning from FK.

Drift is Inevitable

Euler integration errors and wheel slip cause pose estimates to drift. After ~10 m of travel, errors can exceed 10 cm. VO/SLAM (Week 12) corrects this.

Unified Philosophy

FK for mobile base = FK for arm: a deterministic forward function mapping control inputs to predicted configuration. Same math, different geometry.

LAB WEEK 7

MANIPULATOR LABS

Hands-On Labs

Nine progressive lab exercises building FK intuition from scratch — code first, theory second.



LAB 7.1

Planar 2R FK — Manual Implementation

```
import numpy as np
import matplotlib.pyplot as plt

def fk_2r(l1, l2, th1, th2):
    x = l1*np.cos(th1) + l2*np.cos(th1 + th2)
    y = l1*np.sin(th1) + l2*np.sin(th1 + th2)
    return x, y

# Plot workspace
l1 = l2 = 0.5
pts = []
for t1 in np.linspace(-np.pi, np.pi, 50):
    for t2 in np.linspace(-np.pi, np.pi, 50):
        x, y = fk_2r(l1, l2, t1, t2)
        pts.append((x, y))

xs, ys = zip(*pts)
plt.scatter(xs, ys, s=1, alpha=0.3)
plt.axis('equal'); plt.title("2R Workspace"); plt.show()
```

Task

- Implement `fk_2r()` and test with multiple (θ_1, θ_2) pairs
- Generate and plot the full TCP workspace via grid sweep
- Identify the reachable annulus: inner radius $|L_1 - L_2|$, outer $L_1 + L_2$

Duration: ~20 min

Focus on observing how the annular shape emerges from joint limits and link lengths.

LAB 7.2

Planar 2R FK via Robotics Toolbox

```
import roboticstoolbox as rtb
import numpy as np

robot = rtb.models.ETS2.planar2()

# Test configurations
configs = [
    np.array([0.0, 0.0]), # zero configuration
    np.array([0.3, 0.7]), # arbitrary
    np.array([np.pi/4, np.pi/3]),
]

for q in configs:
    T = robot.fkine(q)
    print(f"q = {np.round(q,3)} → TCP = {np.round(T.t[:2], 4)}")
```

Task

- Run `robot.fkine(q)` for each configuration above
- Compare outputs with your `fk_2r()` from Lab 7.1
- Confirm numerical agreement to 4 decimal places

Learning Goal

Bridge between self-written formulas and production toolbox code. Same math — different abstraction level.

Duration: ~15 min

LAB 7.3

Workspace Sweep — Visualizing Reachability

Goal

Scan a fine grid of (θ_1, θ_2) values, compute (x, y) for each, and plot the resulting scatter. Add joint limits to reveal the constrained workspace.

```
th_range = np.linspace(-np.pi, np.pi, 100)
xs, ys = [], []
for t1 in th_range:
    for t2 in th_range:
        x, y = fk_2r(0.5, 0.5, t1, t2)
        xs.append(x); ys.append(y)
```

What to Observe

- Full reachable annulus with $0 \leq r \leq L_1 + L_2$
- Effect of restricting θ_2 in $[0^\circ, 180^\circ]$
- Singular configurations: fully extended and fully folded

Duration: ~20 min

LAB 7.4

6-DOF FK — Panda / Yahboom

```
import roboticstoolbox as rtb
import numpy as np

robot = rtb.models.URDF.Panda()

# Test multiple configurations
configs = {
    "zero (qz)": robot.qz,
    "ready (qr)": robot.qr,
    "random": np.random.uniform(-1, 1, robot.n),
}

for name, q in configs.items():
    T = robot.fkine(q)
    print(f"\n[{name}]\n")
    print(f"  Joints:  {np.round(q, 3)}")
    print(f"  Position: {np.round(T.t, 4)} m")
    print(f"  Rotation: {np.round(T.R, 3)}")
```

 Yahboom 6-DOF uses the same `fkine(q)` call once its URDF or ETS model is imported. Duration: ~20–25 min.

Visualization with Swift — Interactive FK

Steps

1. Launch Swift: `robot.plot(q, backend='swift')`
2. Use the joint sliders to move each DOF interactively
3. Observe the TCP trace path in real time
4. Screenshot at least **two distinct poses** for the lab report

Deliverable

A short paragraph describing what you observe about how individual joint changes affect TCP position and orientation. Duration: ~20 min.

Mobile Robot FK — Lab Overview

01

Lab 7.6 — Unicycle Simulation

Simulate base trajectory from velocity profile $(v(t), \omega(t))$ using Euler integration. Plot the resulting path.

02

Lab 7.7 — Multiple Command Profiles

Test straight, rotate, arc, and S-curve profiles. Connect observed shapes to real robot behavior.

03

Lab 7.8 — Differential Drive FK

Convert (v_L, v_R) to (v, ω) then simulate. Explore different turning radii by varying the ratio.

04

Lab 7.9 — Base + Arm Integration

Simulate base moving to a target while arm holds a fixed FK pose. Prepare intuition for full manipulation pipeline.

LAB 7.6

Unicycle FK — Trajectory Simulation

```
import numpy as np
import matplotlib.pyplot as plt

def unicycle_step(x, y, th, v, w, dt):
    return (x + v*np.cos(th)*dt,
            y + v*np.sin(th)*dt,
            th + w*dt)

# Simulate 100 steps with constant v and omega
x, y, th = 0.0, 0.0, 0.0
v, w, dt = 0.1, 0.2, 0.1 # m/s, rad/s, s
traj = [(x, y)]

for _ in range(100):
    x, y, th = unicycle_step(x, y, th, v, w, dt)
    traj.append((x, y))

xs, ys = zip(*traj)
plt.plot(xs, ys, 'b-o', markersize=2)
plt.axis('equal'); plt.title("Unicycle Trajectory"); plt.show()
```

With constant $v = 0.1\text{m/s}$ and $\omega = 0.2\text{rad/s}$, the robot traces a **circular arc** of radius $r = v/\omega = 0.5\text{m}$. Duration: ~20 min.

LAB 7.7

Exploring Command Profiles

Straight Line

$v > 0, \omega = 0$ Robot moves in the direction of its current heading. Path: straight line. Radius: $r = \infty$.

In-Place Rotation

$v = 0, \omega \neq 0$ Robot spins around its center. Path: a point. Heading θ changes without position change.

Circular Arc

$v > 0, \omega \neq 0$ Radius $r = v / \omega$. Larger $|r|$ = gentler curve. Sign of ω sets CW vs CCW.

S-Curve (Compound)

Alternate $\omega = +0.3$ for 50 steps then $\omega = -0.3$. Produces an S-shaped path — useful for lane changes.

Duration: ~15 min. Sketch expected shapes before running code and verify intuition against simulation output.

LAB 7.8

Differential Drive FK

```
def diff_drive_to_unicycle(v_L, v_R, W):
    """
    Convert differential drive wheel speeds to unicycle model.
    W: wheel track width (distance between wheels in meters)
    """
    v = (v_R + v_L) / 2.0
    w = (v_R - v_L) / W
    return v, w

# Example: left wheel faster → turns right
W = 0.3 # 30 cm track width
v_L, v_R = 0.08, 0.12

v, w = diff_drive_to_unicycle(v_L, v_R, W)
print(f"v = {v:.3f} m/s, omega = {w:.3f} rad/s")
print(f"Turning radius = {v/w:.3f} m")
```

Experiment: set $v_L = 0$, $v_R = 0.1 \rightarrow$ robot rotates around its left wheel. Set $v_L = -v_R \rightarrow$ robot spins in place. Duration: ~20 min.

LAB 7.9

Toy Example: Base + Arm Combined

Scenario

A mobile manipulator must reach a fixed object:

1. Use unicycle FK to drive the base forward 1 m
2. Use arm FK (fixed ready-pose) to compute TCP in base frame
3. Compute TCP in world frame: $T_{\text{world}}^{\text{TCP}} = T_{\text{world}}^{\text{base}} \cdot T_{\text{base}}^{\text{TCP}}$

Why This Matters

This toy example previews the Week 9 full pipeline:

$$T_{\text{world}}^{\text{TCP}} = T_{\text{world}}^{\text{base}} T_{\text{base}}^{\text{arm}} T_{\text{arm}}^{\text{TCP}}$$

No IK needed yet — just FK composition. Shows how base motion changes the world-frame TCP even when arm joints are fixed.

Duration: ~15 min.

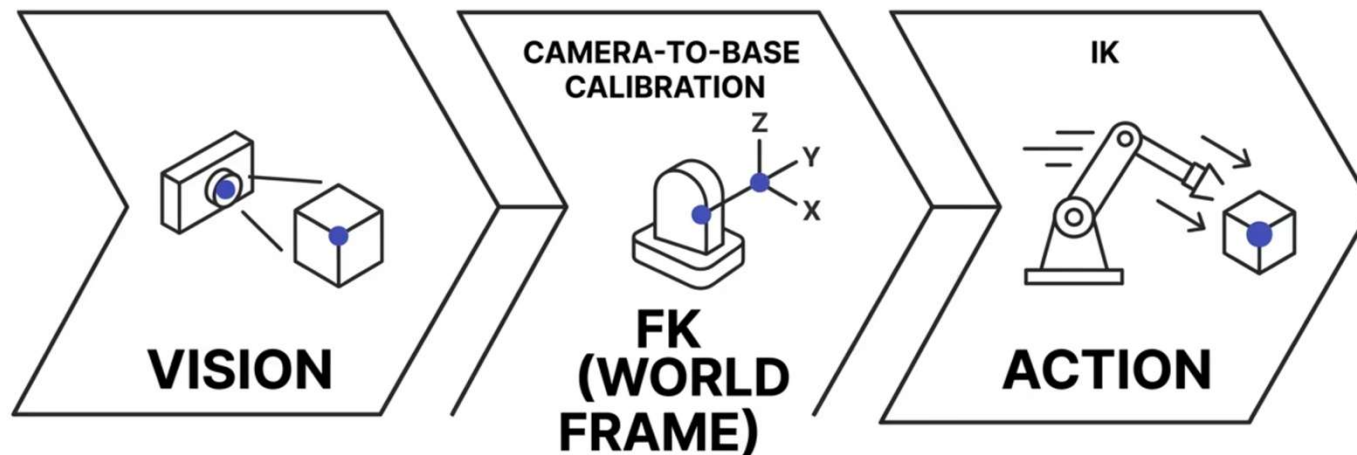
PART G

FK in Context

Connecting Forward Kinematics to perception, control, simulation, and the full robotics pipeline.



FK & Perception: The Bridge



Vision gives the **object's pose relative to the camera**. FK gives the **camera's pose relative to the robot base**. Composing these two transforms produces the object's pose in the base frame — the input needed for IK. This full chain is developed in Week 9.

FK $\leftrightarrow$ IK: The Fundamental Duality

Forward Kinematics (FK)

$$T = \text{FK}(q) : \mathbb{R}^n \rightarrow SE(3)$$

Given joint angles q , compute end-effector pose T . Always has a unique answer. Computationally trivial — just matrix multiplication.

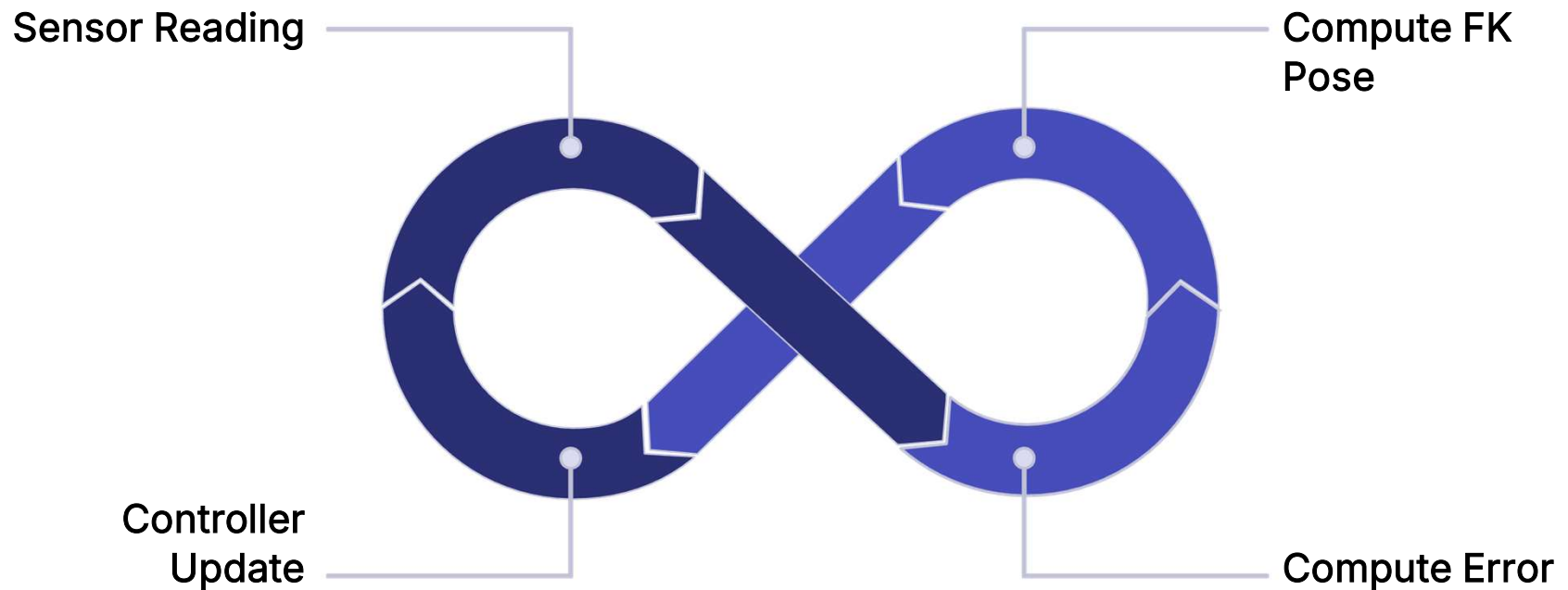
Inverse Kinematics (IK)

$$q = \text{IK}(T) : SE(3) \rightarrow \mathbb{R}^n$$

Given desired pose T , find joint angles q . May have 0, 1, or infinitely many solutions. FK is used to **verify IK solutions**

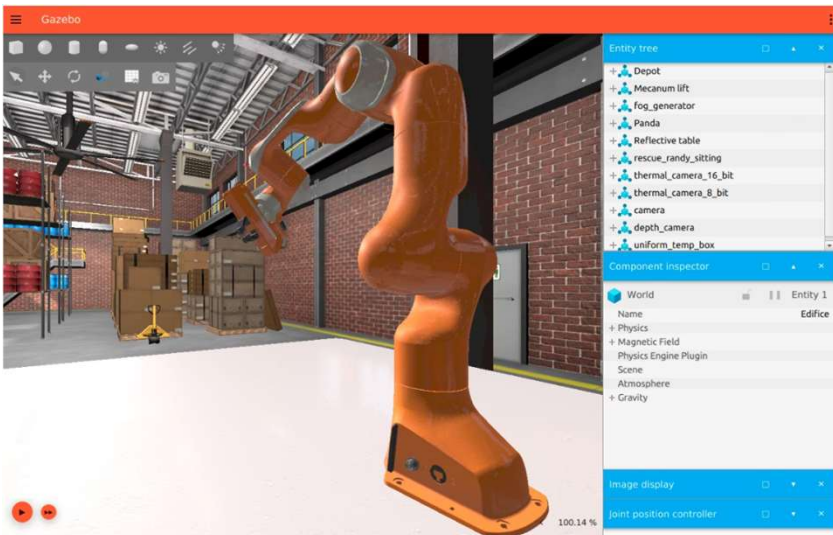
 Week 8 (IK) iteratively adjusts q and calls FK to measure error at each step. Strong FK intuition directly accelerates IK comprehension.

FK in the Control Loop



The control loop runs continuously — typically at 100–1000 Hz for real manipulators. FK must therefore be fast and deterministic. This is why analytic FK (matrix chain) is preferred over iterative methods in real-time control.

FK & Simulation



Simulators Need FK

- Every physics simulator (Gazebo, PyBullet, Swift) uses FK to place each link in 3D space at each timestep
- Collision detection queries bounding volumes derived from FK-computed link poses
- Without FK, the simulator cannot render or simulate the robot

✓ Swift is our course simulator. Every visualization you see in Labs 7.2–7.5 is FK running under the hood at each frame.

Common FK Mistakes

Unit Confusion: Degrees vs Radians

Toolbox functions expect radians. Passing degrees silently produces wrong poses — often very close to correct for small angles, making the bug hard to spot. Always use `np.deg2rad()` when converting.

Unit Confusion: mm vs m

URDF/ETS models typically use meters. CAD models often export in millimeters. A 1000× error in position is easy to miss when working in simulation only.

Frame Confusion

FK returns the TCP pose in the **base frame**, not the world frame. If the robot's base has moved (mobile manipulator), you must compose $T_{\{\text{world}\}^{\{\text{base}\}}} \cdot T_{\{\text{base}\}^{\{\text{TCP}\}}}$.

Wrong Joint Order

A joint vector $q = [q_1, q_2, \dots, q_n]$ must match the model's joint ordering. Swapping two joints produces a numerically valid but physically wrong pose.

How to Debug FK

1 Test Zero Configuration First

Compute `FK(qz)` — for most robots, this should place the TCP directly above the base at a known height. If not, the model is wrong before any joint moves.

2 Move One Joint at a Time

Set all joints to zero, then change q_1 alone. Watch which axis the TCP moves along. Verify this matches the physical robot's first joint axis.

3 Compare Self-Code vs Toolbox

For 2R: run both `fk_2r()` and `rtb.fkine()` on the same input. Values should match to floating-point precision. Any discrepancy reveals a formula error.

4 Print Intermediate Transforms

For multi-DOF arms, compute and print T_{0^1} , T_{0^2} , *ldots*, T_{0^n} separately. Isolate which joint's transform first diverges from expectation.

Preparing for Week 8: Inverse Kinematics

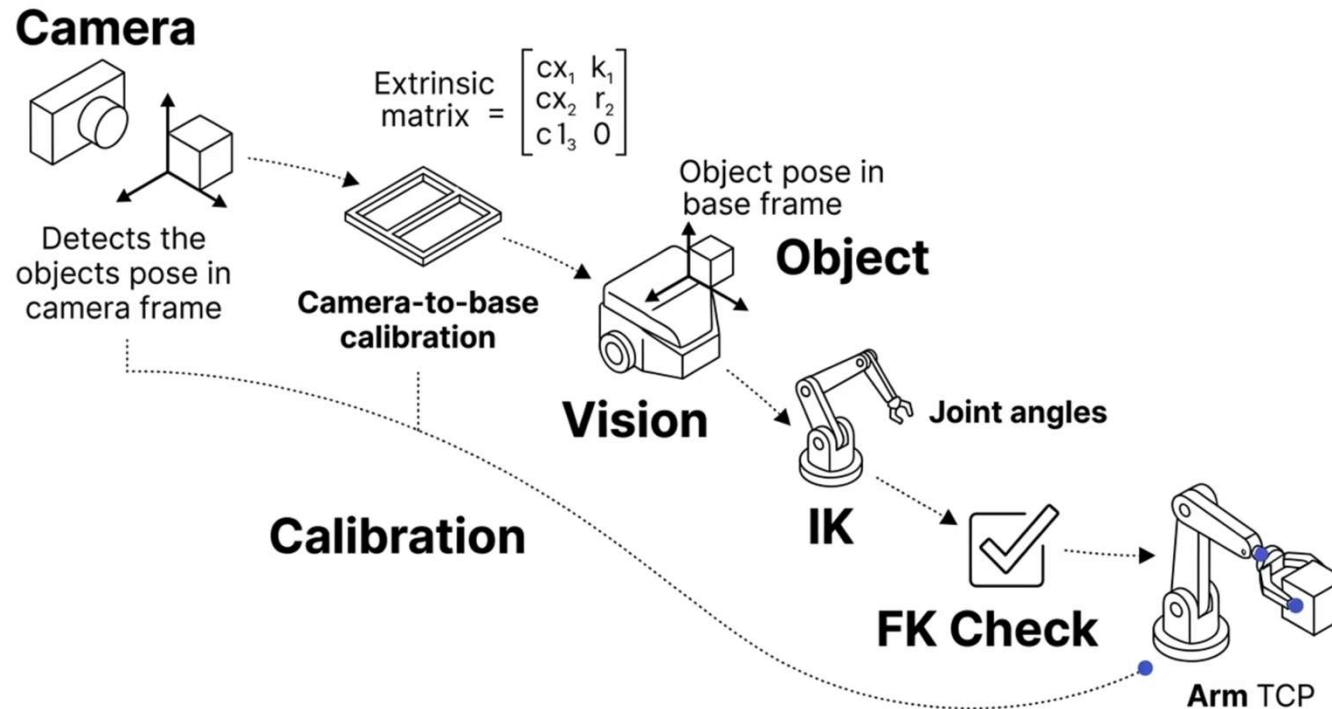
What IK Does

Given a desired TCP pose T^* in $SE(3)$, IK finds joint angles q^* such that $\text{FK}(q^*) = T^*$. Most solvers iterate: adjust q , evaluate $\text{FK}(q)$, compute residual, repeat until convergence.

Checklist Before Week 8

- ☒ Understand FK as a function: inputs, outputs, mathematical form
- ☒ Implement and test `fk_2r()` manually
- ☒ Use `robot.fkine()` for 6-DOF arm
- ☐ Keep FK functions — IK labs will import and reuse them
- ☐ Visualize at least one robot in Swift

Connection to Week 9: Camera-to-Robot Mapping



The entire Week 9 "AI → grasping" pipeline is a chain of FK compositions. Week 7 FK (arm) and Week 8 IK are the engine inside. Calibration provides the camera-to-base transform. Students who build solid FK intuition now will implement this pipeline with confidence.

FK Across the Full Course



Weeks 7–8: Foundation

Build FK for arm and base.
Implement IK using FK as verifier. Labs produce reusable model files.



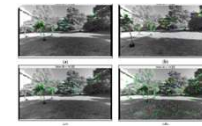
Week 9: Perception Bridge

Compose camera FK with arm FK to map detected object positions into robot-executable commands.



Weeks 10–11: Mobile Base

Apply unicycle and diff-drive FK on real TurtleBot and JetRacer hardware during navigation labs.



Week 12: SLAM & VO

Use FK as the prediction step in VO/SLAM filters. Correct accumulated FK drift with vision feedback.

Key Formulas — Quick Reference

Planar 2R FK

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$$

$$y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

Serial Chain FK

$$T_0^n = \prod_{i=1}^n T_{i-1}^i(\theta_i)$$

PoE FK

$$T(q) = e^{[S_1]q_1} \dots e^{[S_n]q_n} M$$

Unicycle FK (ODE)

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

Diff Drive → Unicycle

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{W}$$

Mobile Manipulator Composition

$$T_W^{TCP} = T_W^B T_B^{TCP}$$

Lab Deliverables Summary

Lab	Task	Deliverable	Time
7.1	Manual 2R FK + workspace plot	Python code + scatter plot	20 min
7.2	Toolbox planar2 fkine	Comparison table: custom vs toolbox	15 min
7.3	Workspace sweep visualization	Workspace scatter with joint limits	20 min
7.4	Panda/Yahboom 6-DOF FK	Printed T matrix for 3 configs	25 min
7.5	Swift interactive visualization	2 screenshots of distinct poses	20 min
7.6	Unicycle trajectory simulation	Trajectory plot + code	20 min
7.7	Multiple command profiles	4 trajectory plots labeled	15 min
7.8	Differential drive FK	Simulation + turning radius analysis	20 min
7.9	Base + arm combined FK	World-frame TCP trace plot	15 min

Week 7 Summary — Part 1: Manipulators

2R Planar FK

From trigonometric formulas to workspace visualization. Each joint contributes an independent circular degree of freedom; the TCP traces arcs in the annular reachable region.

3D Chain via ETS

Serial chains are matrix products. ETS avoids DH frame placement errors and is directly supported by Python Robotics Toolbox — the preferred method in this course.

6-DOF Arms

Panda and Yahboom share the same FK interface: supply $q \in \mathbb{R}^6$, receive $T \in SE(3)$. Position, rotation matrix, Euler angles, and quaternion all accessible from one call.

Week 7 Summary — Part 2: Mobile Robots

Unicycle Model

State (x, y, θ) evolves via ODEs driven by (v, ω) . Euler integration gives discrete FK. Circular arc trajectories emerge from constant inputs.

Differential Drive

Convert (v_L, v_R) to (v, ω) then feed the unicycle model. TurtleBot hardware uses exactly this pipeline in Weeks 10–11.

Drift and Correction

FK integration accumulates pose error over time. VO/SLAM (Week 12) uses perception to correct this drift — but FK remains the prediction step inside the filter.

Core Takeaway: FK as the Universal Prediction Function

Forward Kinematics is the function that tells a robot where it is in the world — given only what it commanded or measured at its joints and wheels. Every higher-level capability in robotics — control, planning, perception integration, grasping — is built on top of this single, fundamental computation.

FK → IK → Control → Perception → Autonomy

Self-Assessment Questions

Conceptual

Can you explain why FK always has a unique answer but IK may have 0, 1, or infinitely many solutions? What geometric property of the configuration space explains this?

Mathematical

For a 3R planar robot with $L_1 = L_2 = L_3 = 0.3$ m, write the FK equations for (x, y) . What is the reachable workspace boundary?

Computational

If `robot.fkine(q).t` returns a position 0.5 m away from where you expect, list 3 possible sources of error and how you would diagnose each.

Applied

A mobile manipulator drives 2 m forward, then the arm FK places the TCP at $(0.3, 0, 0.5)$ in the base frame. What is the TCP position in the world frame?

Reading & Resources for Week 7

Primary Resources

- Python Robotics Toolbox docs — `rtb.models`, `fkine()`, ETS tutorial
- Modern Robotics (Lynch & Park) — Chapter 4: Forward Kinematics
- Robotics, Vision and Control (Corke) — Chapter 7: Robot Arm Kinematics

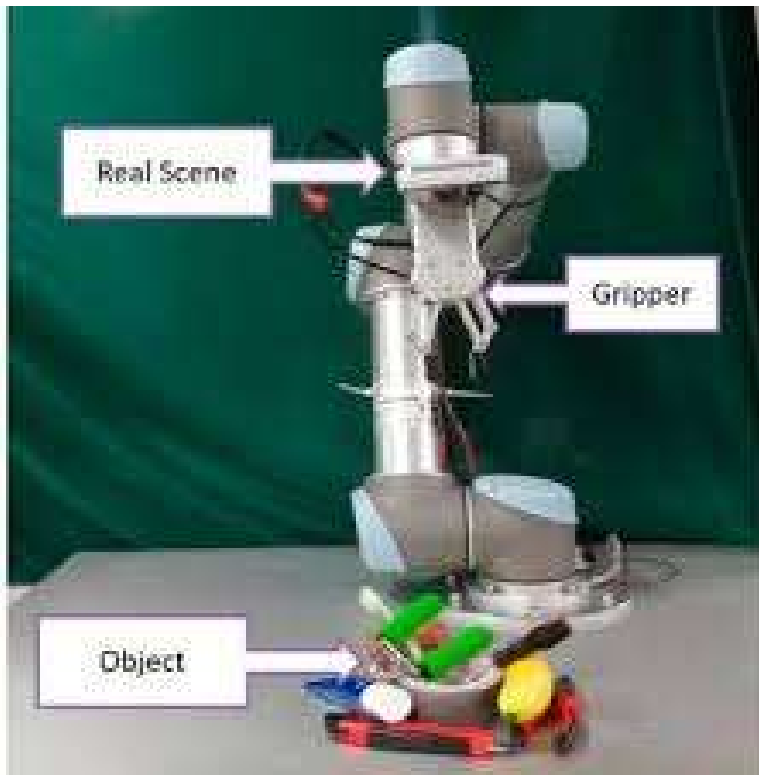
Toolbox Models Used

- `rtb.models.ETS2.planar2()` — Labs 7.1–7.3
- `rtb.models.URDF.Panda()` — Labs 7.4–7.5

Recommended Practice

After completing all labs, revisit Lab 7.9 (Base + Arm). Try to answer: if the object is at world position $(2, 0.3, 0.1)$ m and the base is at $(1.5, 0, 0)$, what FK computation gives you the object's position in the base frame?

This question — and its answer — is exactly what Week 9 builds on.



Looking Ahead: Week 8

Next Week: Inverse Kinematics

We invert the FK function: given a target TCP pose T^* in $SE(3)$, find joint angles q^* such that $\text{FK}(q^*) = T^*$. Your Lab 7 FK functions will be imported directly into IK solvers.

Preparation

- Complete all 9 labs — especially save your FK functions
- Understand why $\text{FK}(q)$ is used as the IK residual checker
- Review the Jacobian concept from Week 6 — it bridges FK to IK algorithms